

Nel contesto della programmazione orientata agli oggetti (OOP), il polimorfismo si riferisce alla capacità di un oggetto di comportarsi in modi diversi a seconda del contesto in cui viene utilizzato. Esistono due tipi principali di polimorfismo

1. Polimorfismo statico (a tempo di compilazione)

Si verifica quando il comportamento di un metodo o di un operatore è determinato a tempo di compilazione. Questo tipo di polimorfismo include:

Overloading (Sovraccarico): Definizione di più metodi o operatori con lo stesso nome, ma con parametri diversi. Si verifica a compile-time (polimorfismo statico).

2. Polimorfismo dinamico (a tempo di esecuzione)

Si verifica quando il comportamento di un metodo è determinato a runtime. Questo è ottenuto tramite:

Ereditarietà e funzioni virtuali.

Utilizzo di puntatori o riferimenti alla classe base per chiamare metodi della classe derivata.

Override: Permette di ridefinire un metodo della classe base in una classe derivata. (Stesso nome e stessa firma (stesso numero, tipo e ordine dei parametri)).

Richiede che il metodo nella classe base sia dichiarato virtual. Si verifica a run-time.

Riepilogo delle differenze		
Caratteristica	Override	Overload
Dove avviene?	Tra classe base e classe derivata.	Nella stessa classe.
Stesso nome?	Sì.	Sì.
Stessa firma?	Sì.	No, deve cambiare.
Quando avviene?	A runtime (polimorfismo dinamico).	A compile-time (polimorfismo statico).

La programmazione orientata agli oggetti (OOP) è un paradigma di programmazione che si basa sull'uso di "oggetti" per organizzare e strutturare il codice. Oggetti e classi sono i concetti fondamentali di questo paradigma.

Classe

Una classe è un modello o una struttura che definisce proprietà (attributi) e comportamenti (metodi) che gli oggetti creati da quella classe avranno in comune.

Oggetto

Un oggetto è un'istanza concreta di una classe. Rappresenta un'entità reale con attributi specifici e comportamenti definiti.

Incapsulamento

L'incapsulamento consiste nel nascondere i dettagli interni di un oggetto e permettere l'accesso solo attraverso metodi pubblici definiti come getter e setter. Questo aiuta a proteggere i dati e a mantenere il controllo sul loro utilizzo. Ovviamente l'incapsulamento è strettamente legato all'ereditarietà.

EREDITARIETA' :

L'ereditarietà nelle classi è un concetto fondamentale nella programmazione orientata agli oggetti (OOP). L'ereditarietà permette di creare nuove classi basate su classi esistenti, favorendo il riutilizzo del codice e una struttura più modulare. Consente a una classe derivata di ereditare attributi e metodi da una classe detta classe base. Vi sono 3 diversi tipi di ereditarietà:

Tipo di ereditarietà	Membri pubblici nella derivata	Membri protetti nella derivata	Membri privati nella derivata
Pubblica	Pubblici	Protetti	Non accessibili
Protetta	Protetti	Protetti	Non accessibili
Privata	Privati	Privati	Non accessibili

private

Gli elementi private sono accessibili solo all'interno della classe stessa. Non possono essere utilizzati direttamente da oggetti della classe o da altre classi.

public

Gli elementi public sono accessibili ovunque, sia all'interno della classe stessa, sia al di fuori di essa (ad esempio, da altre classi o da funzioni globali).

protected

Gli elementi protected sono accessibili: all'interno della classe stessa e dalle classi derivate (subclassi) di quella classe.

L'ereditarietà in C++ è spesso utilizzata in combinazione con il polimorfismo, che consente di chiamare metodi della classe base tramite un oggetto della classe derivata. Per fare ciò, bisogna dichiarare i metodi come virtual nella classe base, e il metodo corretto verrà chiamato in base al tipo reale dell'oggetto

INCAPSULAMENTO:

L'incapsulamento è uno dei principi fondamentali della programmazione orientata agli oggetti (OOP) e consiste nel nascondere i dettagli interni di un oggetto, esponendo solo le informazioni necessarie per interagire con esso. Questo viene fatto per proteggere lo stato dell'oggetto e per permettere di modificarne il comportamento senza influenzare il codice che lo utilizza.

Accesso controllato agli attributi: Gli attributi di una classe sono solitamente dichiarati come private o protected, il che significa che non possono essere modificati direttamente dall'esterno della classe. Per permettere l'accesso a questi dati, vengono forniti dei metodi pubblici (noti come getter e setter).

Astrazione dell'implementazione: L'incapsulamento permette di nascondere la complessità dell'implementazione interna e di esporre solo un'interfaccia semplice per l'utente. Così, l'utente di una classe non deve preoccuparsi di come la classe esegue una determinata operazione, ma può semplicemente utilizzare i metodi pubblici.

L'inserimento in una lista singolarmente linkata ha una complessità che dipende dalla posizione in cui viene effettuato:

Lista con puntatore head:

Inserimento in testa:

La complessità è $O(1)$.

Basta creare un nuovo nodo, farlo puntare all'attuale primo nodo della lista e aggiornare il puntatore della testa (head).

2. Inserimento in coda:

La complessità è $O(n)$, dove n è il numero di nodi della lista.

È necessario scorrere la lista fino all'ultimo nodo per aggiornare il puntatore next di quest'ultimo al nuovo nodo.

-->Se si mantiene un puntatore al nodo finale (tail), la complessità diventa $O(1)$.

3. Inserimento in una posizione specifica:

La complessità è $O(n)$, poiché potrebbe essere necessario scorrere la lista fino alla posizione desiderata.

lista doppiamente linkata:

Inserimento in testa: La complessità è la stessa ($O(1)$).

Inserimento in coda: È più efficiente con una lista doppiamente linkata se si tiene traccia del nodo tail, poiché l'inserimento avviene in $O(1)$. Con una lista singolarmente linkata, senza un riferimento diretto all'ultimo nodo, richiede $O(n)$.

Inserimento in una posizione specifica: La complessità è la stessa in entrambi i casi ($O(n)$), poiché è necessario scorrere la lista per trovare la posizione.

Grafi ~

Un grafo è una coppia (V, E) , dove:

- $V \Rightarrow$ insieme di nodi
- $E \Rightarrow$ insieme degli archi

↳ gli archi sono rappresentati da coppie di nodi (u, v)

Grafo DIRETTO: un grafo è diretto se ogni arco ha una direzione. L'arco uscente dal nodo u (nodo di partenza) è entrante nel nodo v (nodo di arrivo)



* **Adiacenza:** Si verifica quando un arco collega direttamente due nodi.
↳ $(u, v) \in E$

Grafi NON DIREZIONATI: un grafo è non orientato se non c'è distinzione tra il nodo di partenza u e quello di arrivo v per un dato arco (u, v)

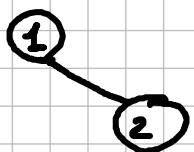
CAMMINO di un grafo: Un cammino è una sequenza di nodi connessi da archi. In un grafo non orientato il cammino può essere percorso in entrambe le direzioni. In un grafo orientato, la direzione degli archi deve essere rispettata.

→ Un **SOTTOCAMMINO** è una sequenza di nodi di un cammino di un grafo.

CICLO: è un cammino chiuso, in cui il primo vertice v_0 e v_k coincidono e non ci sono ripetizioni di nodi. Un grafo senza cicli è detto aciclico.

Grafi non orientati **CONNESSI**:

Un grafo non orientato è connesso se esiste un cammino tra ogni coppia di nodi. Se un grafo ha più di una componente connessa, allora non tutti i nodi sono raggiungibili l'un l'altro. Anche i singoli nodi sono c.c.



③ $\{1, 2\} \{3\}$ 2 c.c.

(non posso raggiungere il 3 dal 1 o dal 2)

Grafo orientato **FORTEMENTE CONNESSO**:

per ogni coppia di nodi esiste un cammino orientato dal nodo di partenza e viceversa.
Se c'è più di una C.F.C. non tutti i nodi sono reciprocamente raggiungibili.

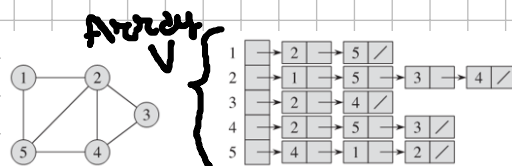
• **SOTTOGRAFO**: è un grafo che contiene un sottoinsieme dei nodi e archi del grafo originario.

• Un grafo (non orientato) è **COMPLETO** se ogni nodo è adiacente.

RAPPRESENTARE I GRAFI

• Liste di adiacenza → utili per i grafi sparsi (nodi > archi). richiedono uno spazio di $O(|V| + |E|)$

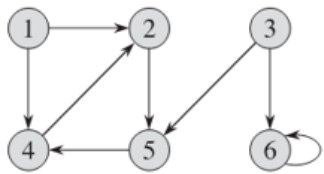
Si usa un array di grandezza V (num. nodi). Ogni elemento è una lista che contiene i nodi adiacenti a quel determinato nodo.



Matrici di adiacenza: rappresentano il grafo con una matrice $|V| \times |V|$. Ogni posizione $A[i][j]$ rappresenta l'assenza o presenza di un arco.

$A[i][j] = 1 \rightarrow$ arco presente

$A[i][j] = 0 \rightarrow$ arco assente



	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

RICERCA IN AMPIEZZA

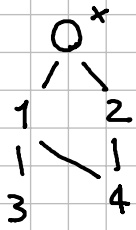
BREATH-FIRST-SEARCH

Algoritmo di esplorazione di grafi. Permette di visitare tutti i nodi raggiungibili a partire da un nodo sorgente.

- 1 Il nodo sorgente viene inserito in una coda e contrassegnato come visitato
- 2 Nella coda si tiene traccia dei nodi visitati, inizialmente impostati come non visitati. (bianchi)
- 3 Si estrae un nodo dalla coda e si visitano i suoi adiacenti (quando si estrae il nodo diventa nero)

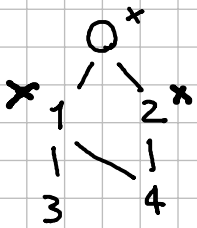
4 Ogni nodo ancora non visitato viene contrassegnato come visitato e aggiunto in coda

• Alla fine, si ottiene un albero di visita che copre tutti i nodi raggiungibili dal nodo di partenza (albero di copertura in ampiezza)



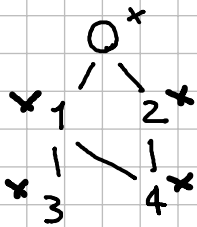
coda
 $\leftarrow 0$

estraggo 0 i suoi adiacenti
sono 1 e 2, li aggiungo alla coda



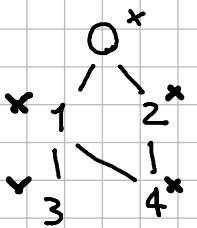
coda
 $\leftarrow 1 \ 2$

estraggo 1, aggiungo 3 e 4
alla coda



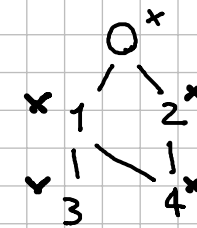
coda
 $\leftarrow 2 \ 3 \ 4$

estraggo 2, il suo adiacente
4 è già stato visitato



coda
 $\leftarrow 3 \ 4$

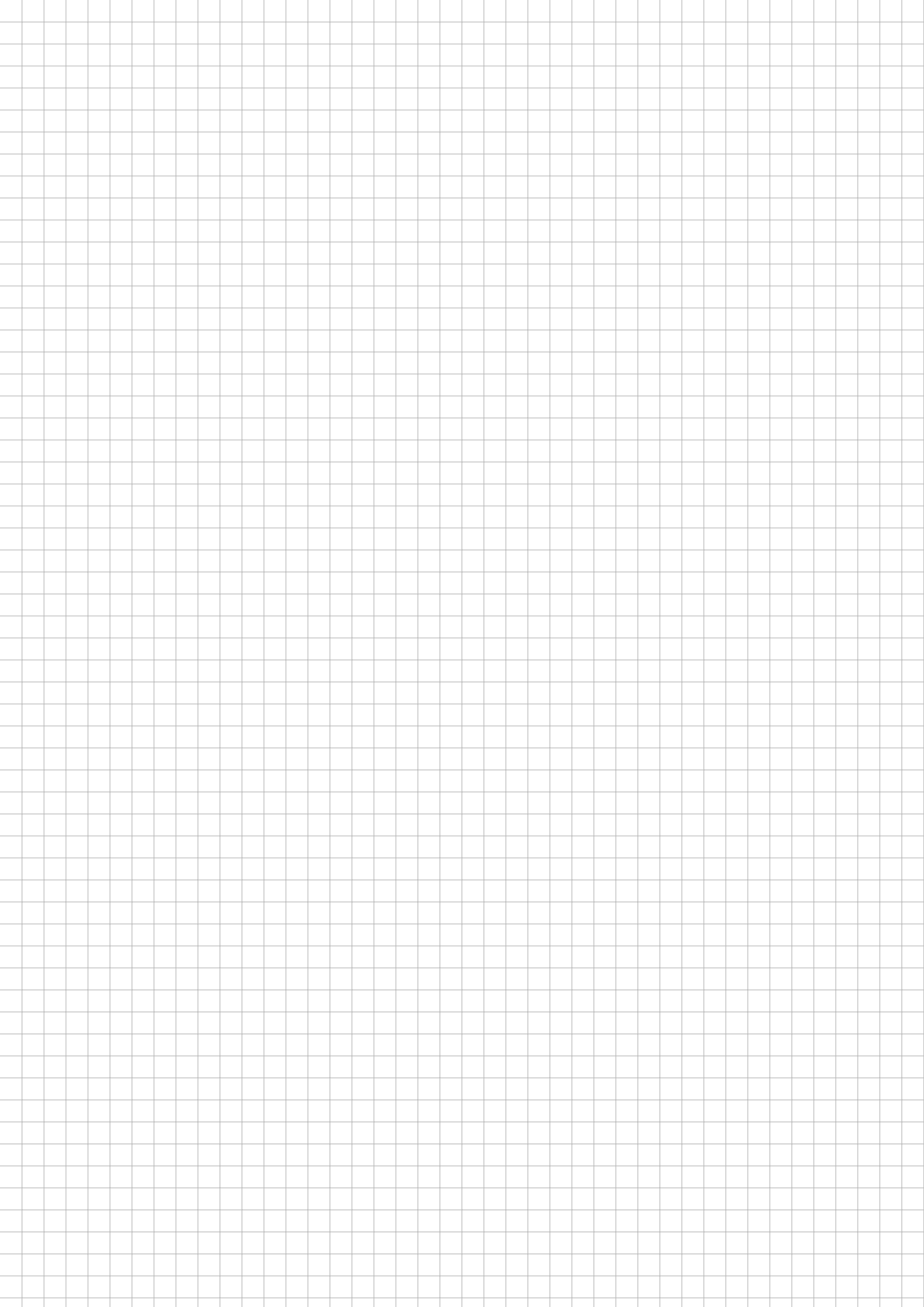
estraggo 3, il suo adiacente
1 è già stato visitato



coda
 $\leftarrow 4$

estraggo 4, i nodi 3, 1, 2 sono
già stati visitati.

COMPLESSITÀ BFS (con liste di adiacenza) =
 $O(V+E)$ = (numero di nodi + numero di archi)



RICERCA IN PROFONDITÀ

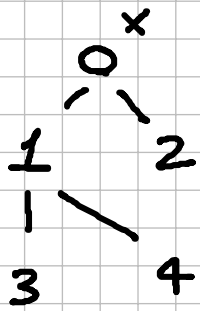
DEPTH FIRST SEARCH

Algoritmo per esplorare grafi. Si basa sull'idea di esplorare il più possibile lungo ogni ramo prima di tornare indietro. non possono esserci cicli?

- 1 Si seleziona un nodo sorgente e lo marchiamo come visitato.
- 2 Manteniamo una lista di nodi visitati per evitare di riesplorare nodi già considerati.
- 3 Controlliamo i nodi adiacenti a quello corrente: se quello adiacente non è stato visitato, questo diventa il nuovo nodo corrente, viene aggiunto alla lista e seguito come visitato.
- 4 Quando si raggiunge un nodo che non ha nodi adiacenti non visitati, si fa backtracking.
↳ Si torna al nodo precedente nella lista di esplorazione e si cercano eventuali nodi non visitati.

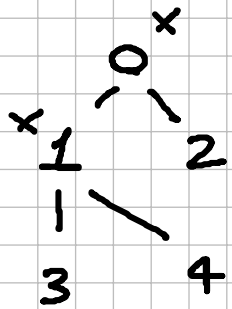
Alla fine, si ottiene un albero di visita che copre i nodi raggiungibili dal nodo di partenza.

↳ (Albero di copertura in profondità)



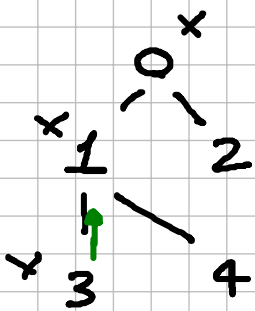
Lista
0

Aggiungo 0 e lo
seguo come nodo
visitato



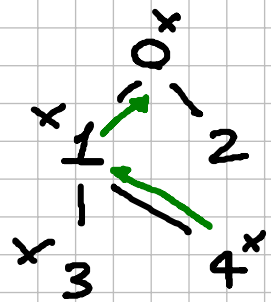
Lista
0 1

Visitiamo 1, adiacente
a 0. Lo aggiungiamo alla
lista.



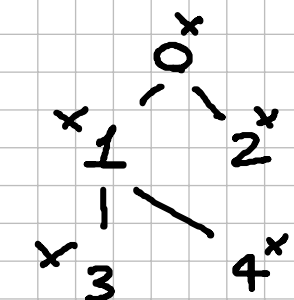
Lista
0 1 3

Il nodo 3 non ha
altri nodi adiacenti.
Facciamo backtracking al
nodo 1.



Lista
0 1 3 4

L'altro nodo adiacente a
1 è 4, lo inseriamo in
lista. 4 non ha altri
adiacenti, facciamo backtracking
dal 4 e poi a 0.



Lista
0 1 3 4 2

L'ultimo nodo da visitare
rimane 2.

codice depth first search

Alberi

- Grafo DIRETTO in cui ogni nodo può avere un solo arco entrante e qualsiasi numero di archi uscenti.

RADICE: L'unico nodo che non ha archi entranti.

FOGLIA: Un nodo che non ha archi uscenti.

Negli alberi non ci sono cicli.

Il nodo da cui parte un arco si chiama padre, il nodo di arrivo si chiama figlio.

Due nodi con lo stesso padre sono fratelli.

Profondità/

Livello: Il livello di un nodo è la sua distanza dalla radice. (numero di archi che devono essere percorsi per arrivare dalla radice al nodo) **Livello = profondità**

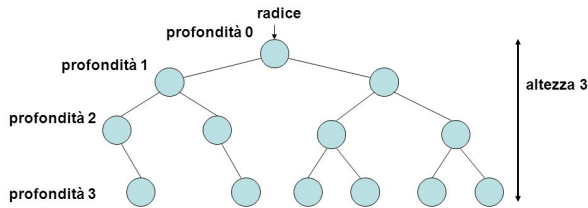
Altezza: L'altezza è la massima profondità tra tutte le foglie di un albero.

```

int Profondità(Nodo* u){
    int p = 0;
    while(u->getParent() != nullptr){ //Risali fino alla radice
        p++;
        u = u->getParent(); //vai al genitore del nodo
    }
    return p;
}

```

Profondità e altezza



Albero equilibrato: Condizioni di equilibrio:

- Tutte le foglie si trovano allo stesso livello (ogni percorso dalla radice a una foglia deve avere lo stesso lunghezza)
- Ogni nodo interno (compreso la radice) deve avere k figli

Alberi Binari

- Ogni nodo può avere al massimo 2 figli, sx e dx. Ad ogni livello n un albero binario può contenere 2^n nodi.
- Il numero totale di nodi di un albero binario è al massimo $2^{(n+1)} - 1$

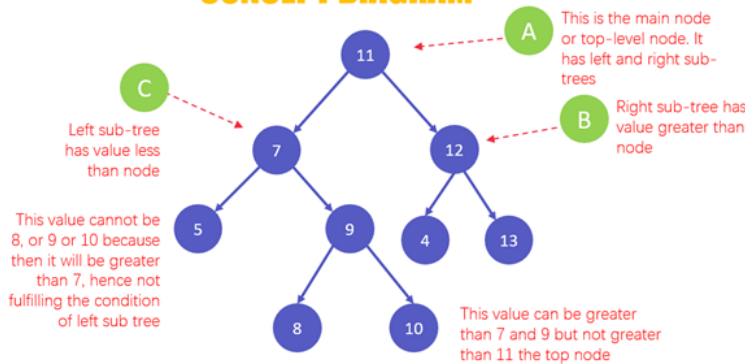
Proprietà fondamentali di un albero

Dato un nodo x

- Figlio sx $< x$
- Figlio dx $> x$

→ Vale anche per i
sottoalberi di x

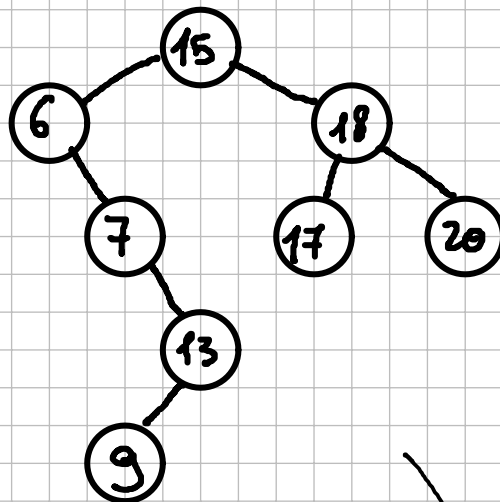
CONCEPT DIAGRAM



Inserimento in un albero binario

I nuovi elementi vengono inseriti come nuove foglie

- Esempio
- Dove metterlo il 9?



dx
perché è più piccolo
del 13, ma più grande
di 7.


```
Insert(Nodo* T, int elemento){
```

```
    Nodo* x = T.root; //x = tmp  
    Nodo* y = nullptr; //y = prev
```

```
    while(x != nullptr){  
        y = x;  
        if(elemento < x->val) x = x->left;  
        else x = x->right;  
    }
```

```
    Nodo* newnode = new Nodo(val);  
    newnode->parent = y;
```

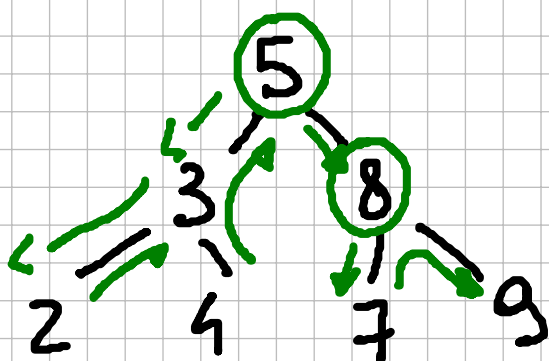
```
    if(y == nullptr) root = newnode;  
    else if (val < y->val) y->left = newnode;  
    else y->right = newnode;
```

```
}
```

Visita di un albero

↳ Complessità $O(n)$ → per tutte le visite!

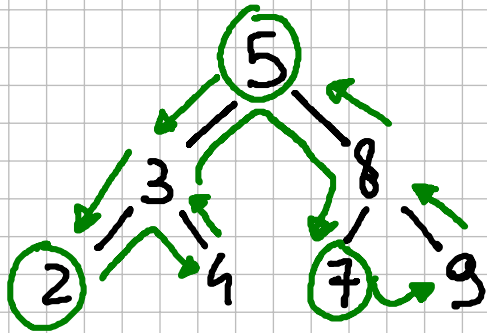
- Preorder: Radice - Sinistra - Destra



5 - 3 - 2 - 4 - 8 - 7 - 9

```
void preOrder(Nodo* p){  
    if(p){  
        cout << p -> val << ' '; //visita la radice  
        preOrder(p -> sinistro); //visita il sottoalbero sinistro  
        preOrder(p -> destro); //visita il sottoalbero destro  
    }  
}
```

- Postorder : Sinistra - Destra - Radice

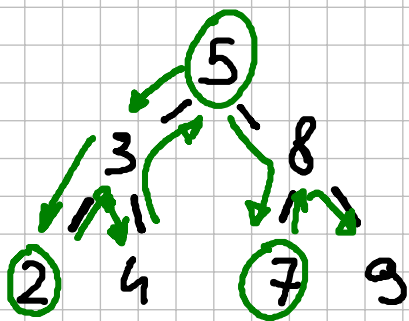


2-4-3-7-8-5

↓
(Il nodo radice viene stampato per ultimo)

```
void postOrder(Nodo* p){
    if(p){
        postOrder(p -> sinistro);
        postOrder(p -> destro);
        cout << p -> val << ' ';
    }
}
```

- In order : Sinistra - Radice - Destra
- Restituisce i nodi in ordine crescente



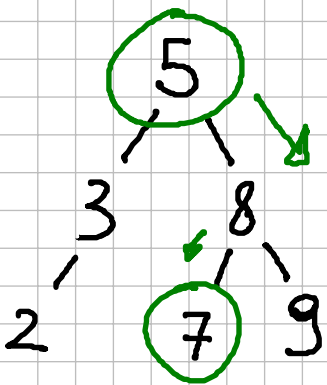
2-3-4-5-7-8-9

```
void inOrder(Nodo* p){
    if(p){
        inOrder(p -> sinistro);
        cout << p -> val << ' ';
        inOrder(p -> destro);
    }
}
```

Successore albero di ricerca

1° caso: Sottalbero destro non vuoto

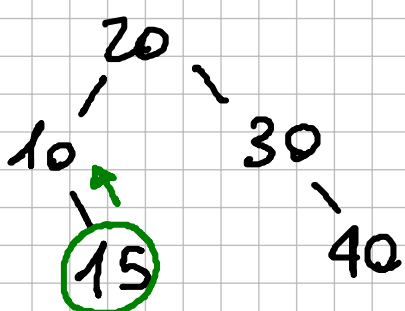
Il successore del nodo x è il nodo più a sinistra del sottalbero destro di x (essendo il minimo)



→ Troviamo il successore di 5. Esiste un sotto albero dx.
Il minimo nel sottalbero dx è 7

2° caso: Sottalbero destro vuoto

Se il nodo x non ha un sottalbero destro si risale verso i genitori di x , si trova il primo genitore p tale che x sia il figlio sx di p . → Se questa condizione è rispettata, il nodo p è il successore.



$p_{10 \rightarrow 15}$ è un figlio dx
continuo a salire

$p_{20 \rightarrow 10}$ è un figlio sx ✓
20 è il successore!!

```

Nodo* Successore(Nodo* x) {
    if (x->right != NULL)
        return Minimo(x->right); // 1o caso (minimo del sottoalbero dx)

    Nodo* y = x->padre; // Risaliamo l'albero
    while (y != NULL && x == y->right) {
        x = y;
        y = y->padre;
    }
    return y; // Successore, o NULL se x è il massimo
}

```

Condizioni del ciclo while:

Se $y == \text{NULL}$, significa che x è il nodo massimo nell'albero (non c'è successore).

La condizione dice: "Continua a risalire l'albero finché x è il figlio destro del suo genitore y e y non è NULL". Quando x diventa il figlio sinistro di y , sappiamo che y è il successore.

Complessità inserimento: albero "normale" $O(n)$
 albero bilanciato $O(\log n)$

Minimo e massimo ricerca

il valore minimo si trova sempre più a sx dell'albero

```

Nodo* minimo(Nodo* x){
    while(x -> left != nullptr){
        x = x -> left;
    }
    return x;
}

```

```

Nodo* massimo(Nodo* x){
    while( x -> right != nullptr){
        x = x -> right;
    }
    return x;
}

```

il valore max si trova sempre più a dx dell'albero

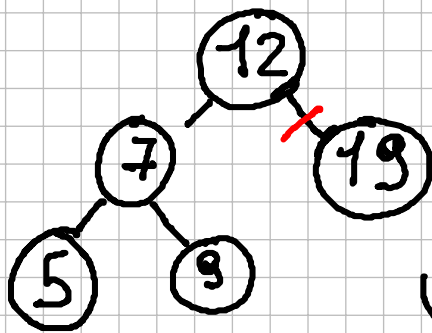
Complessità-
 $O(h) = O(n)$
 $\hookrightarrow h = \text{altezza}$
 albero

Cancellazione di un nodo

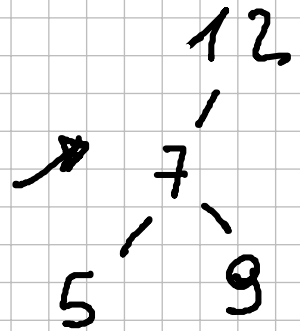
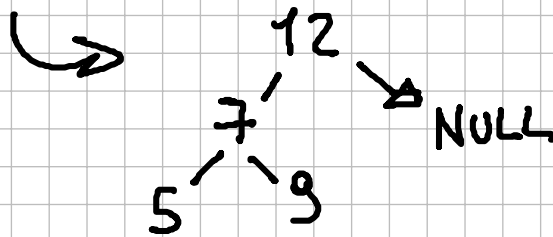
A seconda del numero di figli ci sono 3 casi:

1) Nodo senza figli (foglia):

Si elimina semplicemente il nodo e si imposta il puntatore del padre a NULL.

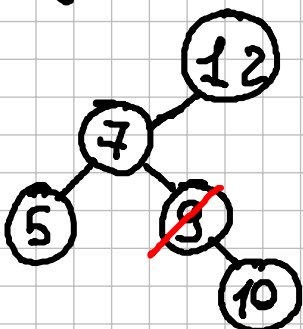


Togliamo il nodo 19
 $19 = z$

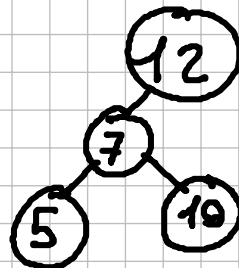


2) Nodo con un figlio

Se il nodo z da eliminare ha un figlio il figlio prende il posto del nodo cancellato



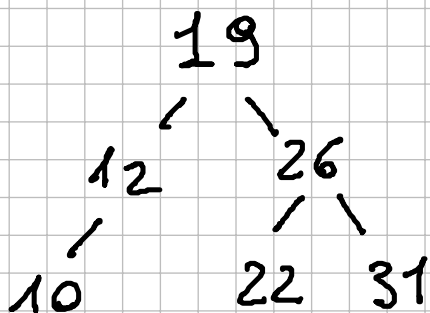
$8 = z$



3) Nodo con due figli

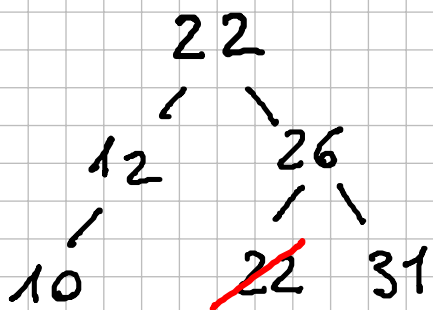
Si eseguono 2 operazioni:

- Si cerca il ^{minimo} nel sottobbero destro e si copia il valore nel nodo da eliminare.
- Si rimuove il nodo che era servito da copia, che sarà un foglio o avrà al max. 1 figlio

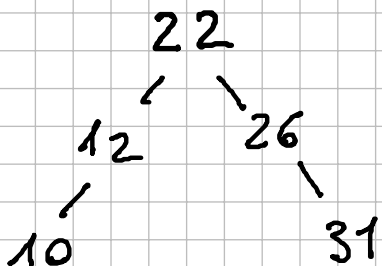


→ Voglio eliminare il 19

- Il 22 è il successore
lo copio al posto
del 19



→ Ora elimino il 22 foglio



→ risultato

Procedura Trapianto

Sostituire un sottoalbero di radice u con un altro sottoalbero di radice v .

Caso 1: u radice dell'albero

Se il nodo u è la radice dell'albero, allora v diventa la nuova radice.

Caso 2: u è figlio sx

Se u è il figlio sinistro di suo padre, il puntatore left del padre di u viene aggiornato per puntare a v che diventa il nuovo figlio sinistro.

Caso 3: u è il figlio destro

Se u è il figlio destro, allora il puntatore right del padre viene aggiornato per puntare a v , che diventa ora il nuovo figlio destro.

```
void Trapianta(Nodo* root, Nodo* u, Nodo* v){
```

```
    if(u->padre == nullptr) { //caso 1
```

```
        root = v;
```

```
    } else if (u == u->padre->sinistro) { //caso 2
```

```
        u->padre->sinistro = v;
```

```
    } else { //caso 3 → u è figlio dx di suo padre
```

```
        u->padre->destro = v;
```

```
    }
```

```
    if(v != nullptr){ //aggiorniamo il padre di v
```

```
        v->padre = u->padre;
```

```
    }
```

```
}
```

*u è figlio sx
di suo padre*

Procedura cancella un nodo

```
void Delete(Nodo* root, Nodo* z){
```

```
    if(z->sinistro == nullptr){ //Se z non ha figlio sx
```

```
        Trapianta(root, z, z->destro); replace z con suo figlio dx
```

```
    } else if (z->destro == nullptr) { Se z non ha figlio dx
```

```
        Trapianta(root, z, z->sinistro); replace z con suo figlio sx
```

```
    } else {
```

*da presidente di essi in cui il nodo z ha sia il figlio dx che il sx
Dobbiamo trovare il successore nel sottoalbero*

Troviamo un valore minimo maggiore di z

```
        Nodo* y = Minimo(z->destro);
```

```
        if(y->padre != z) { se y non è il figlio immediato di z
```

```
            Trapianta(root, y, y->destro);
```

```
            y->destro = z->destro;
```

```
            y->destro->padre = y;
```

```
        }
```

```
        Trapianta(root, z, y);
```

```
        y->sinistro = z->sinistro;
```

```
        y->sinistro->padre = y;
```

```
    }
```

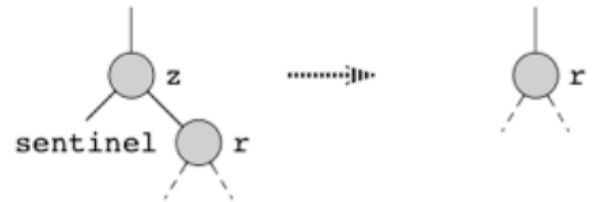
```
    delete z;
```

```
}
```


sentinel = nullptz

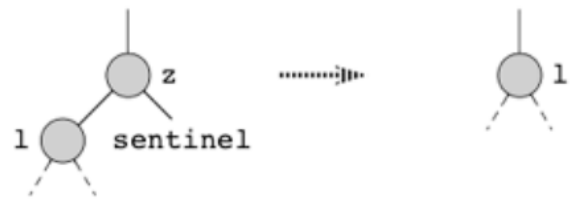
Caso 1:

Se z ha un figlio destro, questo diventa il nuovo nodo al posto di z. (Tramite funzione Trapianta)



Caso 2:

Se z ha un figlio sinistro, questo diventa il nuovo nodo al posto di z. Come nel caso 1, la funzione sostituisce z con il suo figlio usando Trapianta.

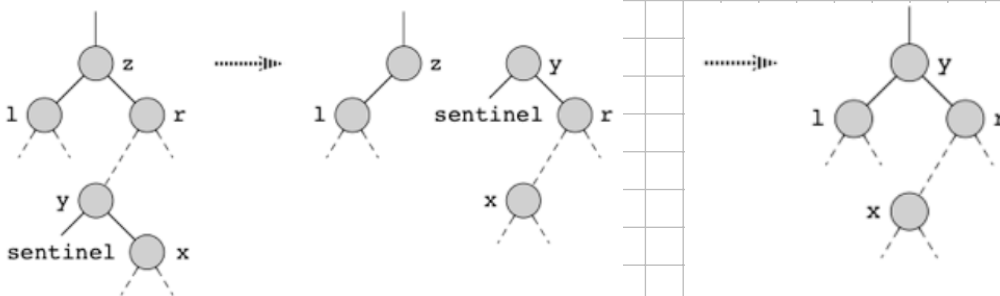


Caso 3 e 4: z ha due figli.

Qui, la situazione è più complessa. Dobbiamo trovare il successore y di z, che è il nodo con la chiave più piccola nel sottoalbero destro di z. Poiché il successore y non ha un figlio sinistro, possiamo "spostare" y nella posizione di z.

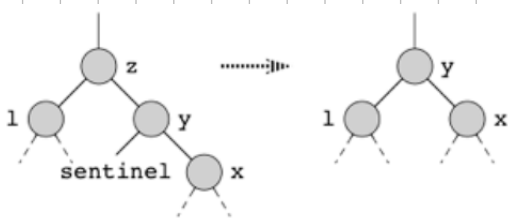
Caso 3:

Se y NON è il figlio destro immediato di z:
In questo caso, dobbiamo prima scambiare y con il suo figlio destro x (se esiste un figlio destro x).
Infine possiamo rimpiazzare y con z



Caso 4:

Se y è il figlio destro immediato di z, scambiamo direttamente y con z. Colleghiamo il figlio sinistro di z a y.



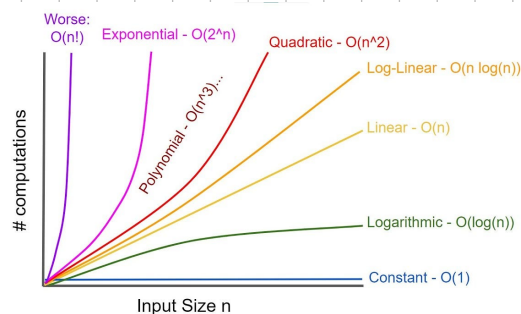
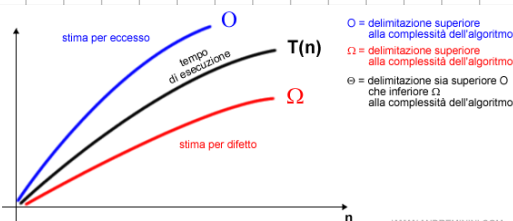
COMPLESSITÀ

- La complessità misura l'efficienza di un algoritmo in termini di risorse consumate, principalmente spazio e memoria:
- Complessità temporale $\rightarrow$ quanto tempo impiega un algoritmo a terminare in funzione della grandezza dell'input.
- Complessità spaziale $\rightarrow$ Quantità di memoria necessaria per eseguire l'algoritmo (esclusa quella usata per l'input).

Complessità Temporale

Neve stante la grandezza dell'input n sia un numero fisso, ci sono diversi fattori (come la disposizione dei dati) che distinguono 3 casi per la complessità temporale:

- BEST CASE $\rightarrow$ input ideale, costo minimo
- AVERAGE CASE, costo mediato
- WORST CASE, costo massimo



Analisi della complessità di crescere di n (analisi asintotica)

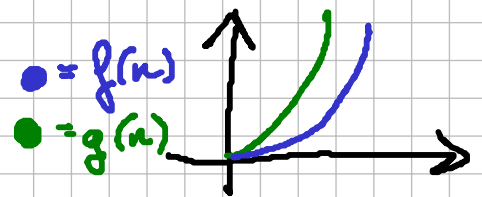
1 O-grande ($O(f(n))$) - Limite superiore

Descrive un limite superiore per la crescita di una funzione.

Se $f(n) \in O(g(n))$

↳ $f(n)$ cresce almeno come g o più lentamente (cioè, non la può superare)

$$0 \leq f(n) \leq c \cdot g(n)$$



2 Omega-grande ($\Omega(f(n))$) Limite inferiore

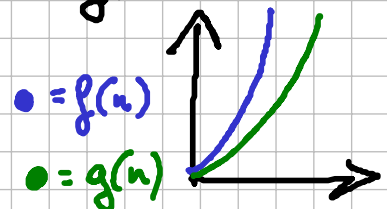
Descrive un limite inferiore per la crescita di una funzione. Se

Se $f(n) \in \Omega(g(n))$

↳ $f(n)$ cresce almeno come g o più velocemente.

(cioè la funzione o è uguale o supera g)

$$0 \leq c \cdot g(n) \leq f(n)$$



Theta - grande ($\Theta(f(n))$) - Limite stretto

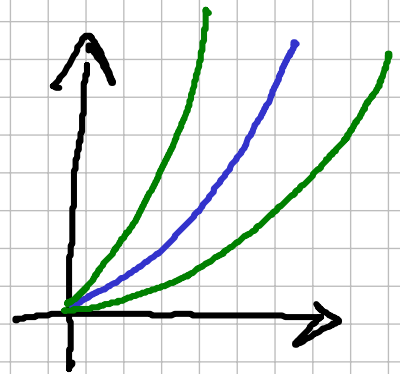
Descrive una crescita proporzionale alla funzione descrivendo un limite stretto.

Se $f(n) \in \Theta(g(n))$

↳ $f(n)$ cresce esattamente come $g(n)$, né più velocemente, né più lentamente.

$f(n)$ deve essere compreso tra $c_1 g(n)$ e $c_2 g(n)$

$$c_1 g(n) \leq f(n) \leq c_2 g(n)$$



Limite Superiore: $O(t(n))$ è un limite superiore per il tempo di esecuzione di un algoritmo. Il tempo non supererà mai $c \cdot t(n)$, dove c è una costante.

Limite inferiore: Se si può dimostrare che nessun algoritmo può risolvere il problema in meno di $t(n)$, allora $\Omega(t(n))$ è un limite inferiore (crescita minima tempo esecuzione).

Algoritmo ottimo: un algoritmo è ottimo se il limite superiore e inferiore coincidono.

↳ Il problema ha complessità $\Omega(f(n))$ e l'algoritmo ha complessità $O(f(n))$

↳ complessità $\Theta(t(n))$

ANALISI DI ALGORITMI

- $O(1) \rightarrow$ complessità di una funzione o blocco di istruzioni, che non contengono cicli, ricorsione ecc...
- $O(n) \rightarrow$ complessità ciclo con variabili incrementate o decrementate di una quantità costante

`for (int i=0; i<=n; i+=c)`

- $O(n^c) \rightarrow$ complessità di cicli annidati
 $c =$ quante volte i cicli for vengono ripetuti

$\rightarrow$ `for (int i=0; i<=n; i+=10) {  $O(n)$`
 $\rightarrow$ `for (int j=0; j<=n; j+=10) {  $(n)$`
`}`
`}`
 $2 \text{ for} \rightarrow O(n^2)$

- $O(\log n) \rightarrow$ complessità quando le variabili sono incrementate o decrementate tramite una moltiplicazione o divisione costante.

`for (int i=0; i<=n; i*=c) / /=c`

- $O(\log \log n) \rightarrow$ complessità ciclo con variabili incrementate / decrementate esponenzialmente

```
for (int i = 0; i < n; i = pow(i, c))
```

```
for (i = n/2; i <= n; i++)  $O(n)$  somma costante
```

```
for (j = 1; j <= n; j = 2 * j)  $O(\log n)$ 
```

```
for (k = 1; k <= n; k = k * 2)  $O(\log n)$ 
```

moltiplicazione

Le complessità vengono moltiplicate quando vi sono cicli annidati.

Se i cicli sono indipendenti, le complessità vengono sommate

In questo esempio, trattandosi di 3 cicli annidati, il risultato è:

$$\hookrightarrow O(n \log^2 n)$$

Se i cicli non fossero stati annidati, avremmo dovuto eseguire una somma delle complessità

$$O(n) + O(\log n) + O(\log n)$$

Il risultato sarà la complessità con la crescita più lenta. $O(n)$

Algoritmi di ordinamento

- Merge Sort → $O(n \log n)$
 - $\log n \rightarrow$ divisione
 - $n \rightarrow$ merge

↳ è basato sul principio "dividi et impera"

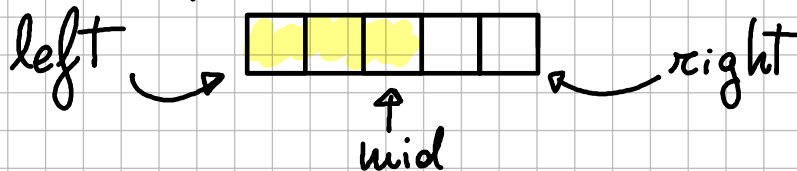
Dividi:

L'array viene diviso a metà in due sotto-array

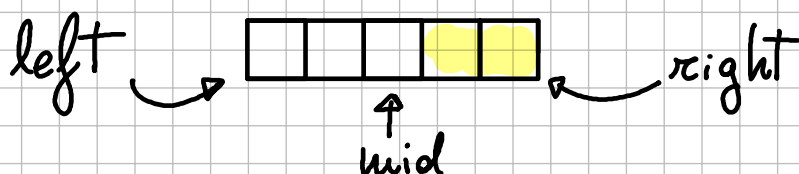
↳ La divisione viene ripetuta ricorsivamente fino ad ottenere sotto-array di un solo elemento, che per definizione sono già ordinati.

↳ L'operazione viene effettuata calcolando il punto medio e chiamando ricorsivamente mergeSort su ciascuna metà

- La prima metà va da left a mid



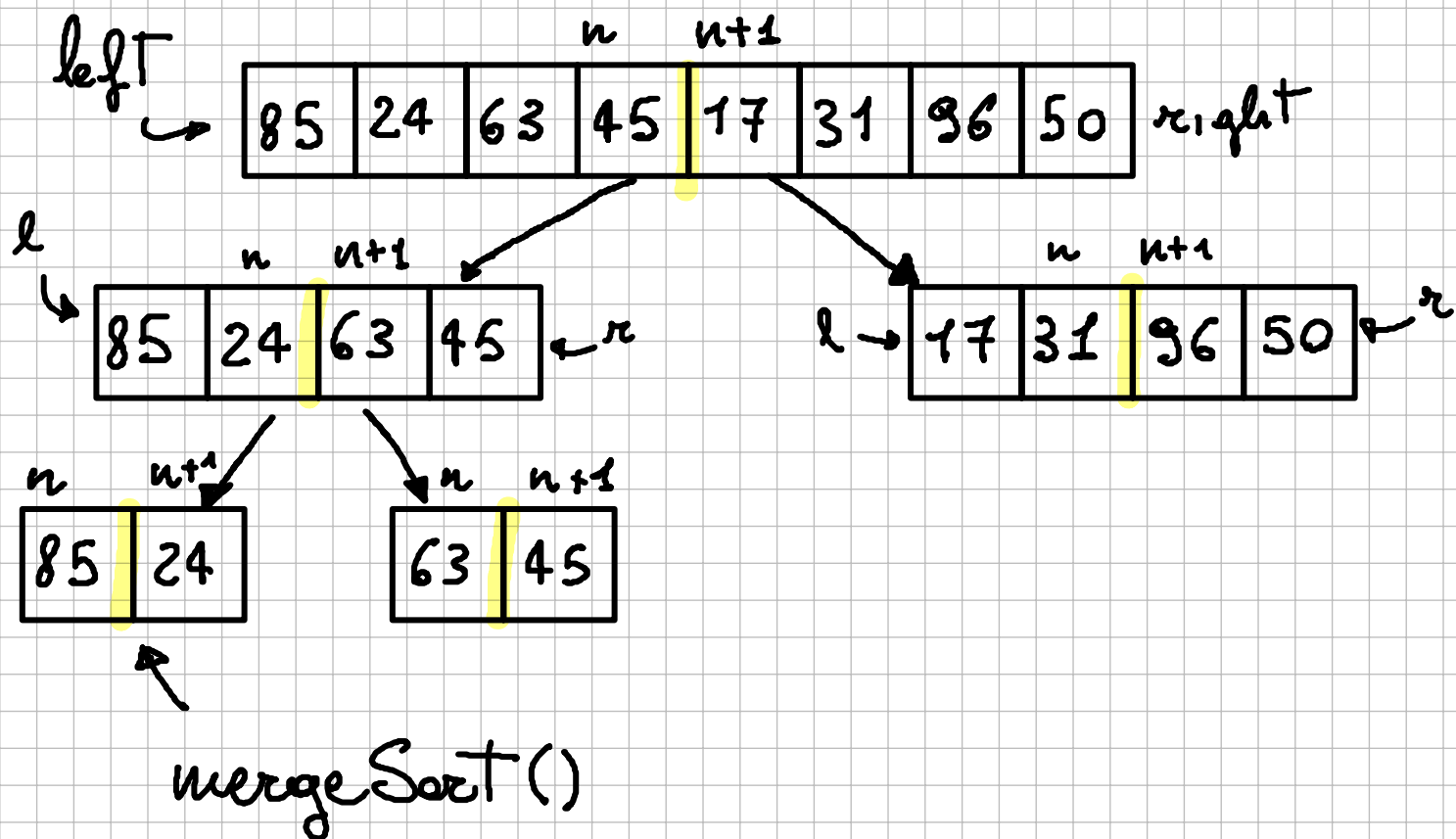
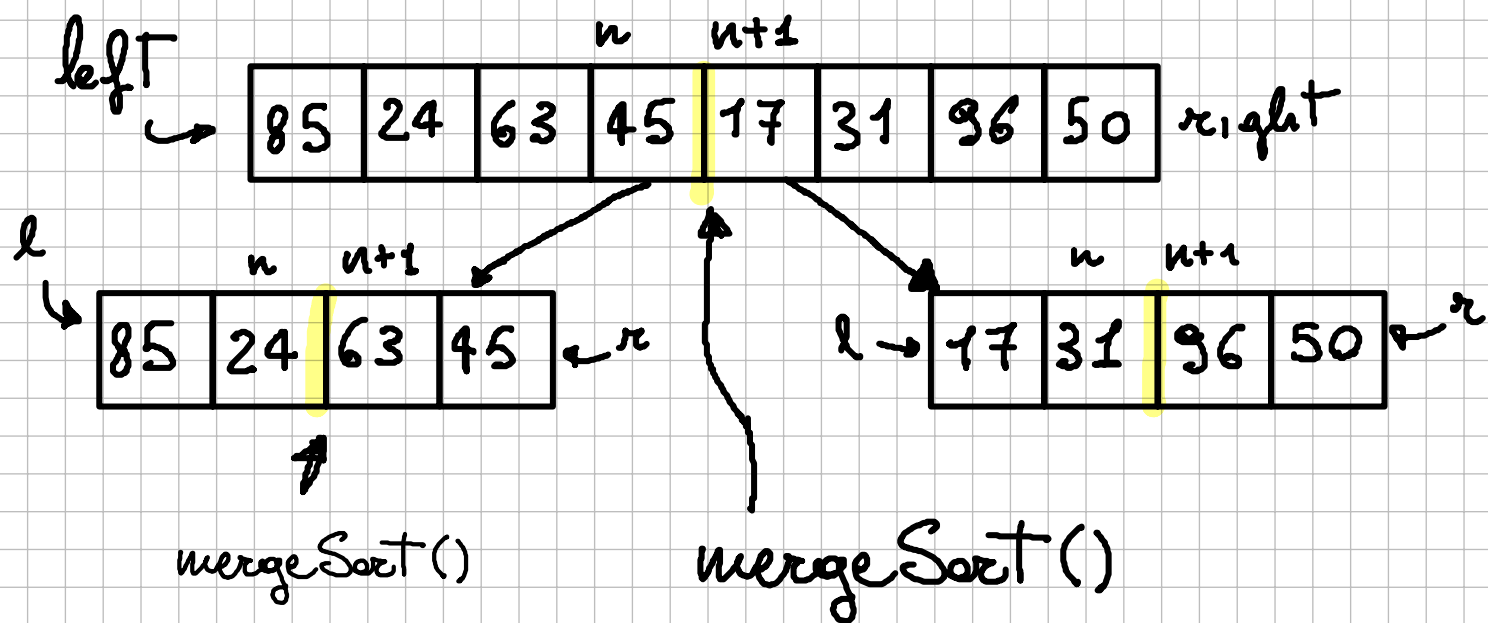
- La seconda metà va da mid+1 a right

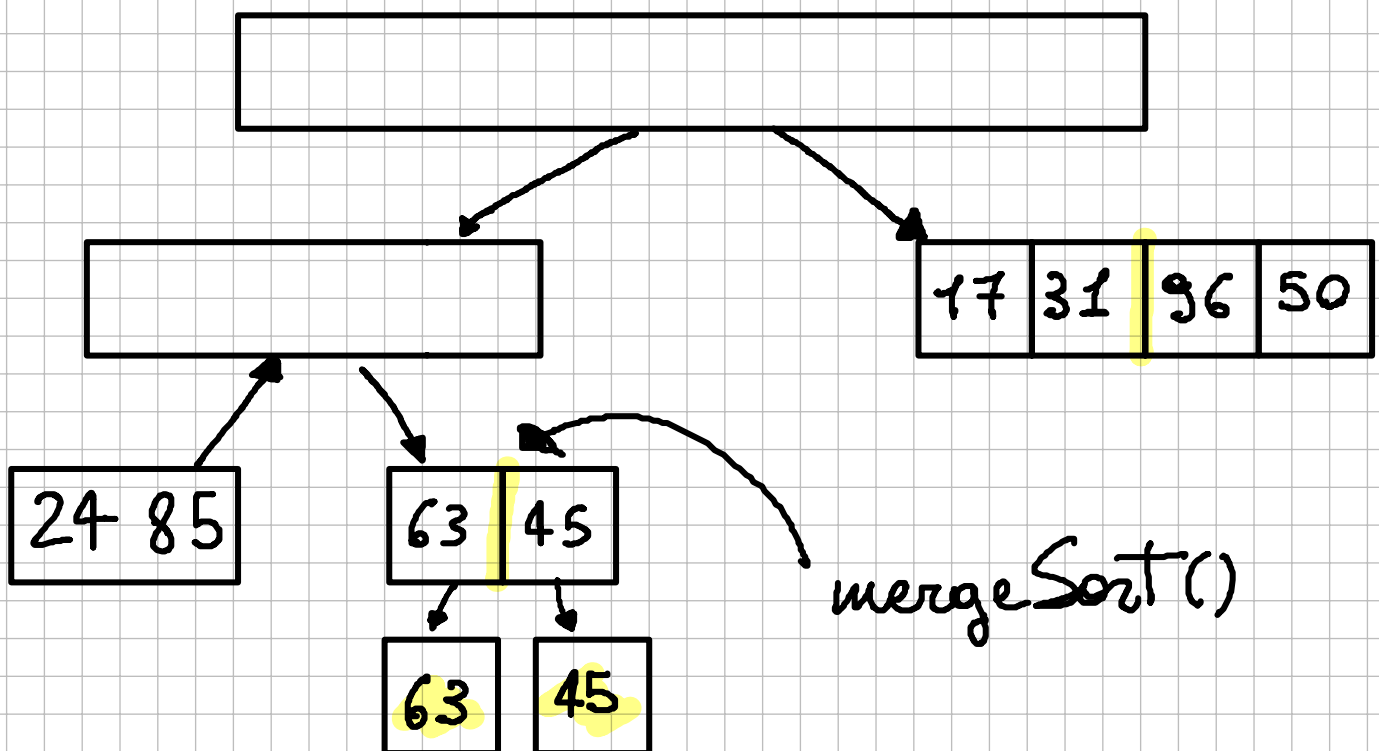
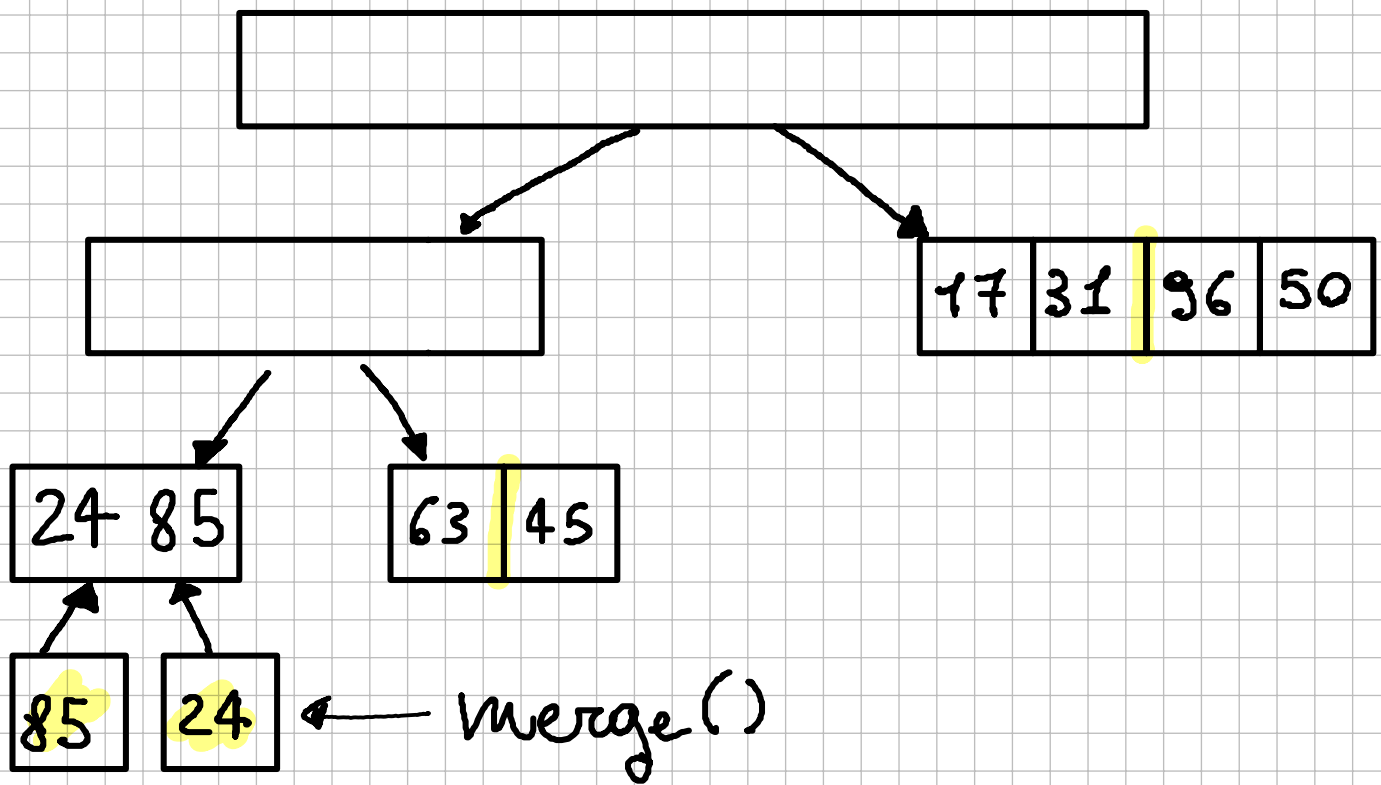


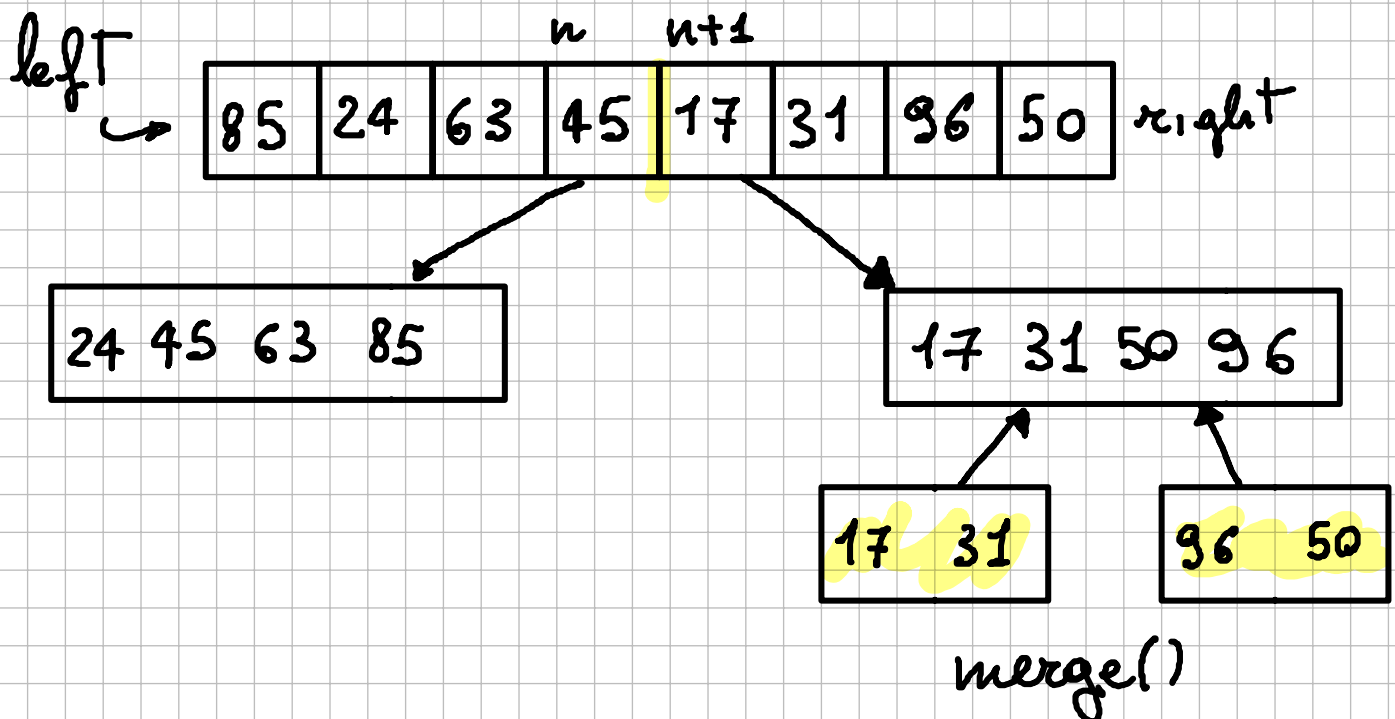
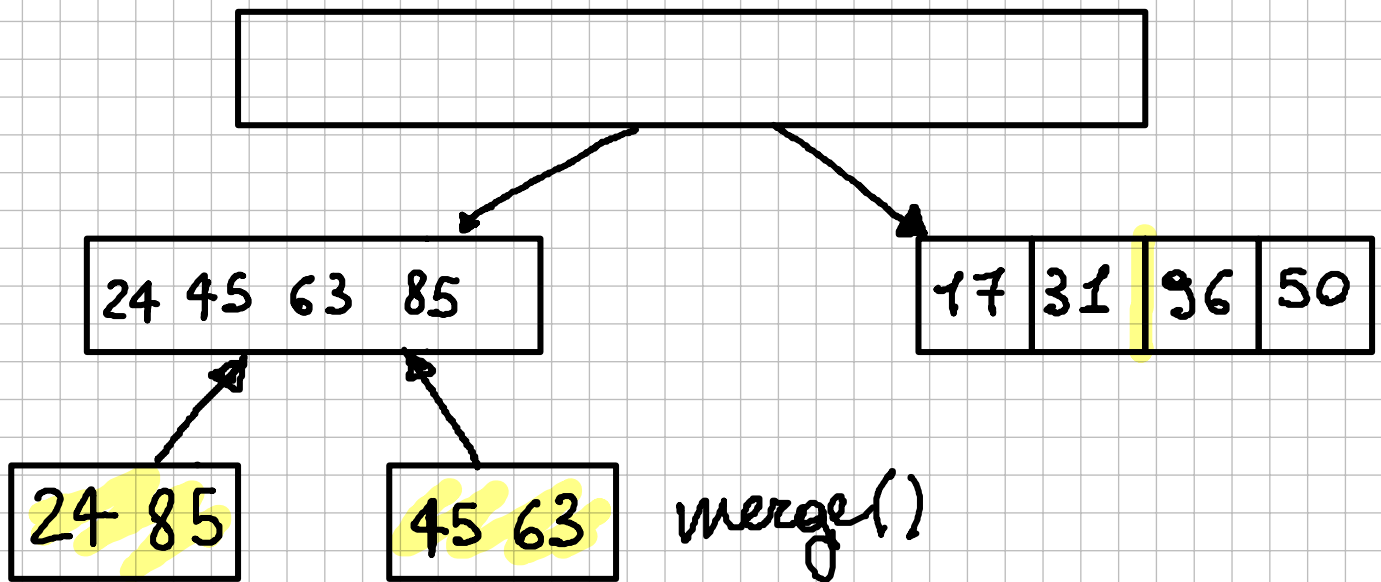
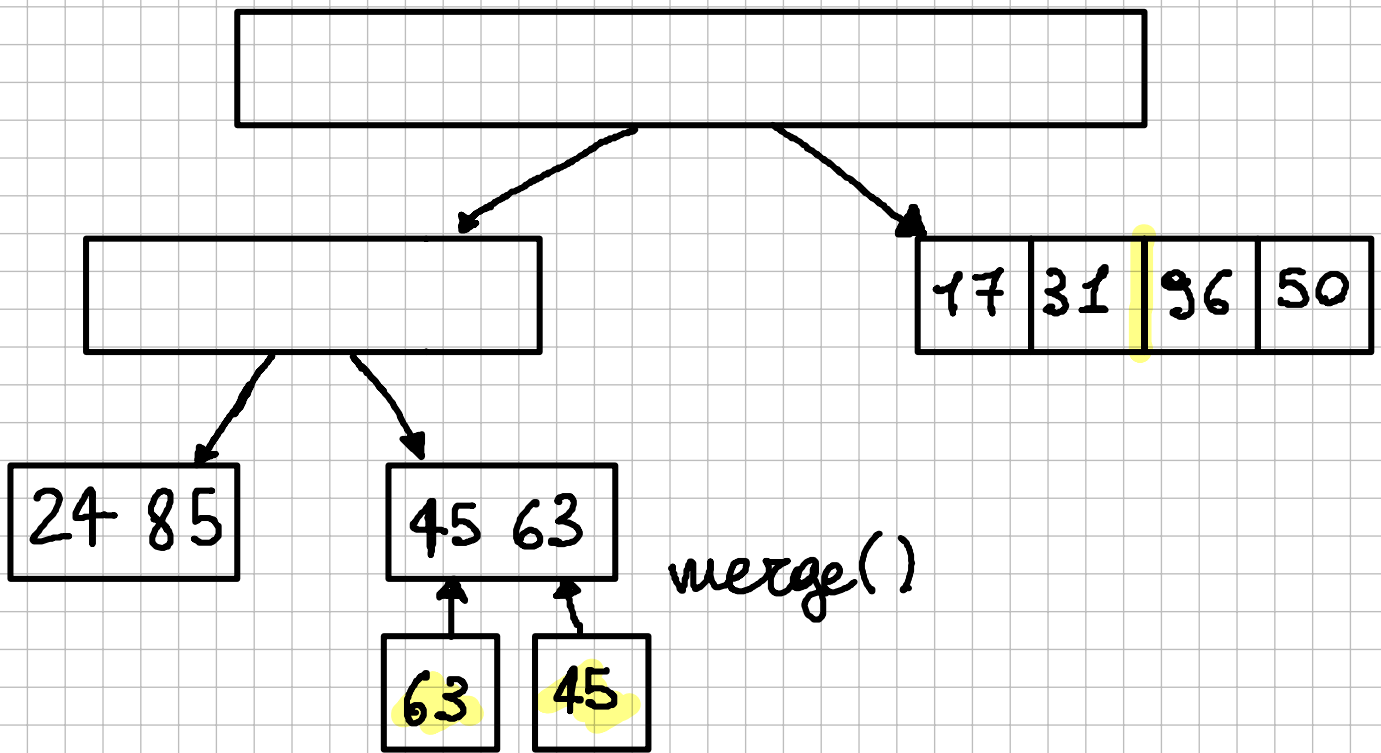
(mid+1 perché mid è già ordinato)

Impero:

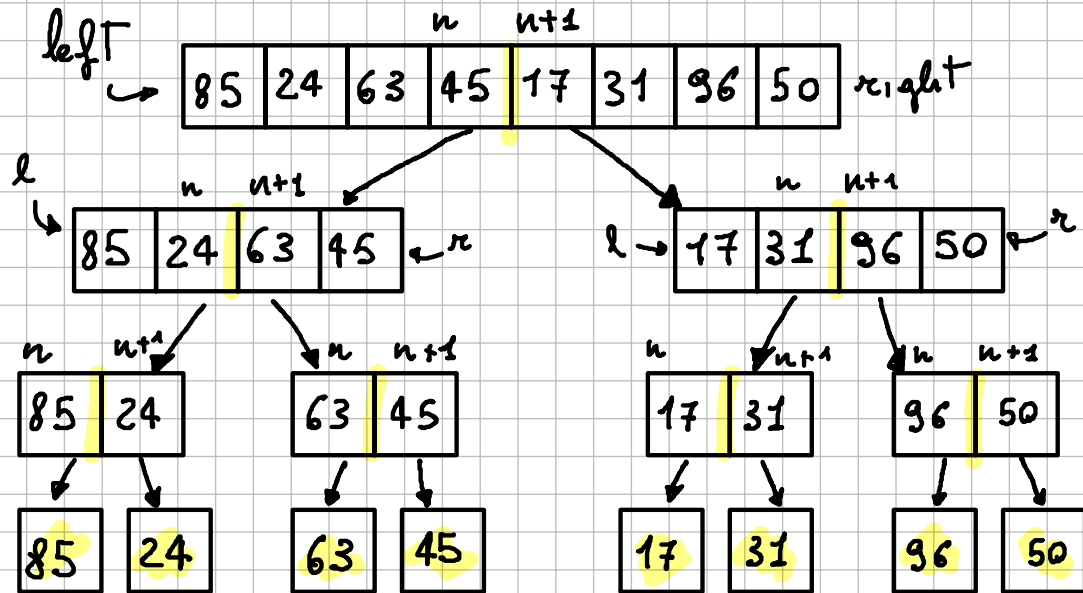
Una volta che i sotto-vettri hanno una cella si inizia il merge. Vengono prima confrontati gli elementi e poi inseriti in un nuovo vettore in modo ordinato.





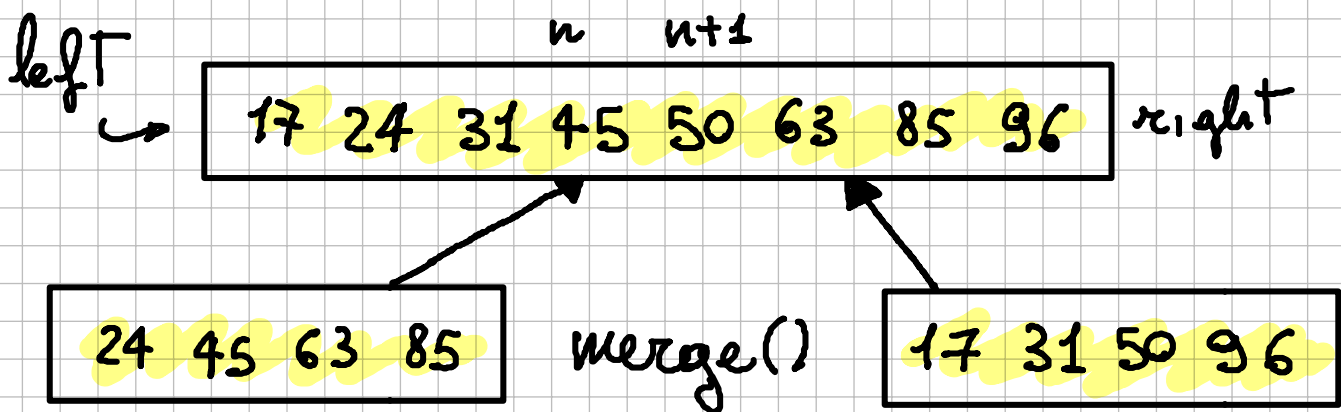


// ho saltato un po' di passaggi
grafici del lato dx
perché sono uguali
a quelli del lato sx



(mergeSort divisione)

RISULTATO



Complessità:

migliore = medio = peggiore = $O(n \log n)$

Sempre uguale perché l'algoritmo divide
con tempo $O(\log n)$ la sequenza e la unisce
in tempo lineare

```
void merge(el v[], int p, int q, int r){
```

prende in input l'array, l'indice iniziale (p) e quello finale (r) dell'array v
n = numero elementi del nuovo array, dato dall'indice iniziale, quello finale + 1

```
    int n = r - p + 1;
    el* v2 = new el[n];
    int i = p; //indice primo sottoarray
    int j = q + 1; //indice secondo sottoarray
    int k = 0; //indice iniziale array temporaneo
```

```
    while ((k < n) && (i <= q) && (j <= r)){
```

con queste condizioni si scano i sotto array per fare i confronti

```
        if (v[i] < v[j]){
            v2[k] = v[i];
            i++;
        } else {
            v2[k] = v[j];
            j++;
        }
        k++;
```

se il valore del sottoarray di sx è più piccolo di quello di dx, allora mettilo nel nuovo sottoarray.
altrimenti, se il valore più piccolo appartiene al sottoarray di dx, metti nel nuovo sottoarray quello

il valore di k viene incrementato in entrambi i casi

```
    }
    while(i <= q){
        v2[k] = v[i];
        i++;
        k++;
    }
    while(j <= r){
        v2[k] = v[j];
        j++;
        k++;
    }
```

cicli per inserire nei nuovi sottoarray eventuali elementi rimanenti (che sono ordinati)

sx

dx

```
    memcpy(v+p, v2, n * sizeof(el));
    delete [] v2;
}
```

```
void mergeSort(el v[], int p, int r){
```

Se ci sono 0 oppure 1 elemento (caso base)

```
    if (r <= p)
        return;
```

exi dalla ricorsione

```
    int q = (p + r) / 2;
```

q = punto medio dell'array

```
    mergeSort(v, p, q);
```

chiamata ricorsiva per la parte sx dell'array

```
    mergeSort(v, q+1, r);
```

chiamata ricorsiva per la parte dx dell'array

```
    merge(v, p, q, r);
```

fusione dei sottoarray che inizia solo quando entrambe

```
}
```

Quick Sort $\rightarrow O(n \log n)$

↳ basato sul paradigma "divide et impera"

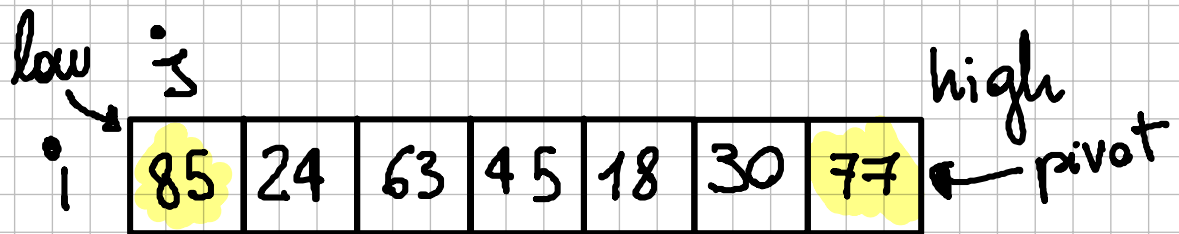
- Scelta del PIVOT: per convenzione prendiamo l'ultimo elemento
- Partition: Divisione dell'array in due sotto parti
 - Parte sx contiene gli elementi minori uguali di pivot
 - Parte dx contiene elementi maggiori di pivot
- Ricorsione: Quicksort() viene richiamato sui suoi due sotto array risultanti dal partizionamento
 - ↳ partition() a ogni chiamata sposta pivot nella posizione corretta nell'array
 - La ricorsione si ferma quando le partizioni hanno lunghezza 0 oppure 1
- Si confronta il pivot con l'elemento in array[i]

Se $array[i]$ è maggiore del pivot:

- si aumenta solo i

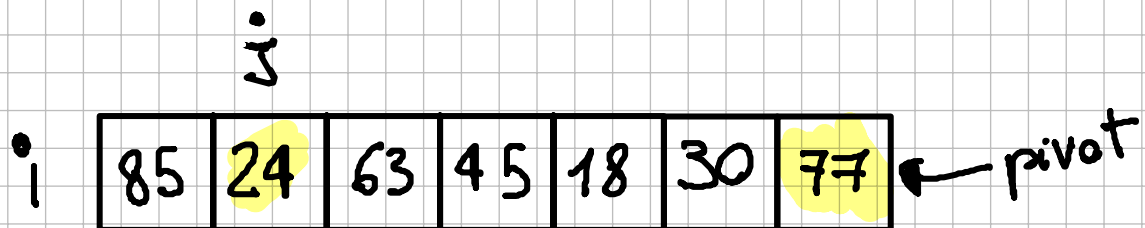
Se $array[i]$ è minore del pivot:

- si aumenta i
- swap $array[i]$ e $array[j]$
- si aumenta j



$i < \text{pivot?} = 85 < 77? \rightarrow \text{NO}$

- Aumento solo i



$i < \text{pivot?} \rightarrow 24 < 77? \rightarrow \text{SI!}$

- Aumento $i++$

i j

85	24	63	45	18	30	77
----	----	----	----	----	----	----

← pivot

- Scambio $V[i]$ e $V[j]$

i j

24	85	63	45	18	30	77
----	----	----	----	----	----	----

← pivot

- Aumento $j++$

i j

24	85	63	45	18	30	77
----	----	----	----	----	----	----

← pivot

i	j						
24	85	63	45	18	30	77	← pivot

$j < \text{pivot?}$ **5!!**

• $i++$

	i	j					
24	85	63	45	18	30	77	← pivot

• Swap i e j

	i	j					
24	63	85	45	18	30	77	← pivot

• $j++;$

	i		j				
24	63	85	45	18	30	77	← pivot

	i		j				
24	63	85	45	18	30	77	← pivot

$j < \text{pivot?}$ **✓**

• $i++$

			i	j			
24	63	85	45	18	30	77	

• Swap i e j

		i	j				
24	63	45	85	18	30	77	

• $j++$

		i		j		
24	63	45	85	18	30	77

			i	j			
24	63	45	85	18	30	77	← pivot

$j < \text{pivot?}$ ✓

• $i++$

			i	j			
24	63	45	85	18	30	77	

• Swap i e j

			i	j			
24	63	45	18	85	30	77	

			i	j			
24	63	45	18	85	30	77	

			i	j			
24	63	45	18	85	30	77	← pivot

$j < \text{pivot?}$ ✓

• $i++$

			i	j			
24	63	45	18	85	30	77	

• Swap i e j

					i	j	
24	63	45	18	30	85	77	

• $j++$

				i	j		
24	63	45	18	30	85	77	

				i		j	
24	63	45	18	30	85	77	← pivot

Siamo arrivati al caso in cui $j = \text{pivot}$ (nel codice equivale alla condizione di uscita del while ($j < r$) dove r è il pivot

Essendo $j < \text{pivot}$, cioè $77 < 77$ non vero eseguiamo uno swap del pivot e l'elemento successivo ad i , array $[i+1]$

				i	i+1	j	
24	63	45	18	30	77	85	← pivot

- Il pivot si trova nella sua posizione corretta
 - gli elementi a sx sono più piccoli
 - gli elementi a dx sono più grandi

Quindi adesso, chiameremo un'altra volta

Quicksort(); mandiamo come primo indice della partizione l'indice iniziale della sottosezione sinistra (quindi p), e come secondo indice l'ultimo elemento spostato a sx del pivot (quindi i)

p e r definiscono rispettivamente l'inizio e la fine della porzione di array che viene ordinata. i rappresenta l'indice finale della porzione sinistra dopo il partizionamento.

```
quickSort(v,p,i); *  
quickSort(v,i+2,r);
```

p		i				
24	63	45	18	30	77	85

```
void quickSort(int v[], int l, int r){
```

int pivot = v[r]; $\rightarrow i$, passato come r quindi il pivot diventa 30

```
int i = l-1;  
int j = l;
```

$l-1$ i		l j				
24	63	45	18	30	77	85

$i \leftarrow \text{pivot}$

quickSort(v, 0, 4); $\leftarrow$ si risolve prima la chiamata di sx

- quickSort(v, 6, 6);

- quickSort(v, 0, 4) partiziona con pivot 30 chiamando a sx (v, 0, 1) e dx (v, 3, 4)

24	18	30	45	63
----	----	----	----	----

↳ Ora risolviamo $(v, 0, 1)$

24 18 $\leftarrow$ pivot

che si completerà con le chiamate
 $(v, 0, -1)$ e $(v, 1, 1)$, che terminano subito

- Ora si può risolvere la chiamata di dx
 $(v, 3, 4)$ 45 63

(a sua volta con le sub chiamate) //



Solo una volta risolte tutte le
subchiamate della prima partizione
a sx, si può procedere con la
ricorsione di dx

↳ quickSort $(v, 6, 6)$

formato solo da 85

$p = \text{inizio array}$ $r = \text{fine array}$

```
void quickSort(int v[], int l, int r){  
    if (r <= l) ← se ci sono 0 o 1 elementi  
        return;  
  
    int pivot = v[r];  
  
    int i = l-1;  
    int j = l;  
  
    while (j < r){ //non metto = perchè non voglio scambiare pivot con se stesso  
        if (v[j] <= pivot){  
            i++;  
            swap(v[i], v[j]);  
        }  
        j++;  
    }  
    swap(v[i+1], v[r]);  
  
    quickSort(v, l, i); parte sx  
    quickSort(v, i+2, r); parte dx  
}
```

`quickSort(v, i+2, r):`

`i+2`: è l'indice iniziale della sottosezione destra dell'array (la porzione a destra del pivot). Questo corrisponde all'indice successivo al pivot, dato che il pivot si trova ora in posizione `i+1`.

Complexity:

peggiore:

Se le partizioni sono formate da 0 e $n-1$, allora la partizione costa $O(n)$. Quindi il caso peggiore ricorsivo è $O(n^2)$

medio:

anche con ripartizioni sproporzionate ad ogni livello, quicksort viene eseguito in $O(n \log n)$. Questo avviene perché qualsiasi ripartizione con proporzionalità costante produce una ricorsione di profondità $O(\log n)$

migliore:

$O(n \log n)$, si verifica quando i due sottoproblemi hanno dimensione $n/2$.

Insertion Sort $\rightarrow O(n^2) / O(n \log n)$

↳ Divide l'array in due parti:

- Inizialmente, il primo elemento è considerato ordinato
- Gli altri elementi sono nella parte non ordinata

Elemento chiave:

Per ogni elemento successivo (dal 2°)

↳ Si prende una Key dalla parte non ordinata.

Confronto elementi ordinati:

↳ Si confronta questo elemento con ciascuno degli elementi nella parte ordinata, da destra verso sinistra

- Gli elementi ordinati che sono maggiori della Key vengono spostati di una posizione a destra verso sinistra

Inserimento:

Quando si trova il primo elemento minore o uguale alla Key (oppure è all'inizio dell'array) si inserisce la Key in quella posizione.

j	i					
85	24	63	45	18	30	77

1° iterazione

Key = array[i] = 24

$j = 0$

↳ $j \geq 0$ && $85 > 24$ (condizione while)

Essendosi verificate entrambe le condizioni si entra nel while

- L'85 va una posizione in avanti

	85	63	45	18	30	77
--	----	----	----	----	----	----

$array[j+1] = array[j]$

- j viene decrementato di 1.

- Usciti dal while, la Key viene inserita nel posto corretto, ovvero $array[j+1]$

↳ Il valore di j quindi è passato da 0 $\rightarrow$ -1

perciò la cella in cui inseriamo la Key è $(-1) + 1 = 0$

$$key = j+1$$

24	85	63	45	18	30	77
----	----	----	----	----	----	----

	j	i				
24	85	63	45	18	30	77

$$key = array[i] = 63 \quad j = 1$$

$$85 > 63? \quad \checkmark$$

$$array[j+1] = array[j]$$

$$85 \text{ va avanti di 1} \quad j--; j = 0$$

	j	i				
24		85	45	18	30	77



Key = 63 $j = 0$

$24 > 63$? X si esce dal while

↳ si va direttamente a

array $[j+1] = \text{Key}$

	$j+1$	i				
24	63	85	45	18	30	77

		j	i			
24	63	85	45	18	30	77

Key = 45

$j = 2$

j
 $85 > 45$? ✓

85 va avanti

$j--; = 1$

	j					
24	63		85	18	30	77

$63 > 45$? ✓

63 va avanti

$j--; = 0$

	j					
24		63	85	18	30	77

24 > 45 ? $\times$

si esce dal while $j = 0$

quindi $array[j+1] = key$

$j+1$

24	45	63	85	18	30	77
----	----	----	----	----	----	----

e così via ...

```
void insertionSort(int* array, int n){  
    for(int i = 1; i < n; i++){  
        int key = array[i];  
        int j = i-1;  
  
        while(j >= 0 && array[j] > key){  
            array[j+1] = array[j];  
            j--;  
        }  
  
        array[j+1] = key;  
    }  
}
```

Complessità

migliore: $O(n)$

medio: $O(n^2)$

peggiore: $O(n^2)$

Attenzione!

Si può migliorare l'algoritmo
usando la ricerca binaria

Riducendo la complessità da $O(n^2)$ a $O(n \log n)$,
perché riduce il numero di confronti per trovare
la posizione corretta nella quale mettere l'elemento
 $array[i]$. Con questo metodo, la ricerca impiega
 $O(\log n)$ + lo spostamento degli elementi $O(n)$
 $\hookrightarrow O(n \log n)$

	Caso Migliore	Caso Medio	Caso Peggior
BubbleSort	$O(n^2)$	$O(n^2)$	$O(n^2)$
BubbleSort + flag	<u>$O(n)$</u>	$O(n^2)$	$O(n^2)$
	Caso Migliore	Caso Medio	Caso Peggior
InsertionSort	$O(n)$	$O(n^2)$	$O(n^2)$
	Caso Migliore	Caso Medio	Caso Peggior
MergeSort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
	Caso Migliore	Caso Medio	Caso Peggior
QuickSort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$

con ricerca binaria:
 $O(n \log n)$

Selection sort: Costo $O(n^2)$ in qualsiasi caso

come funziona l'allocazione dinamica della memoria?

-Nel linguaggio C++ si usa la keyword "new" per allocare memoria dinamica e "delete" per deallocarla.

differenza tra allocazione dinamica e statica.

Le variabili dinamiche vengono allocate nell'heap, mentre le variabili statiche nello stack.